

**Course: Fundamentals of Generative AI and Large Language
Models: Theory and Practice**

**Week 2: Autoencoders: The Representation
Learning Model**

Today's Session Overview

✓ Autoencoders

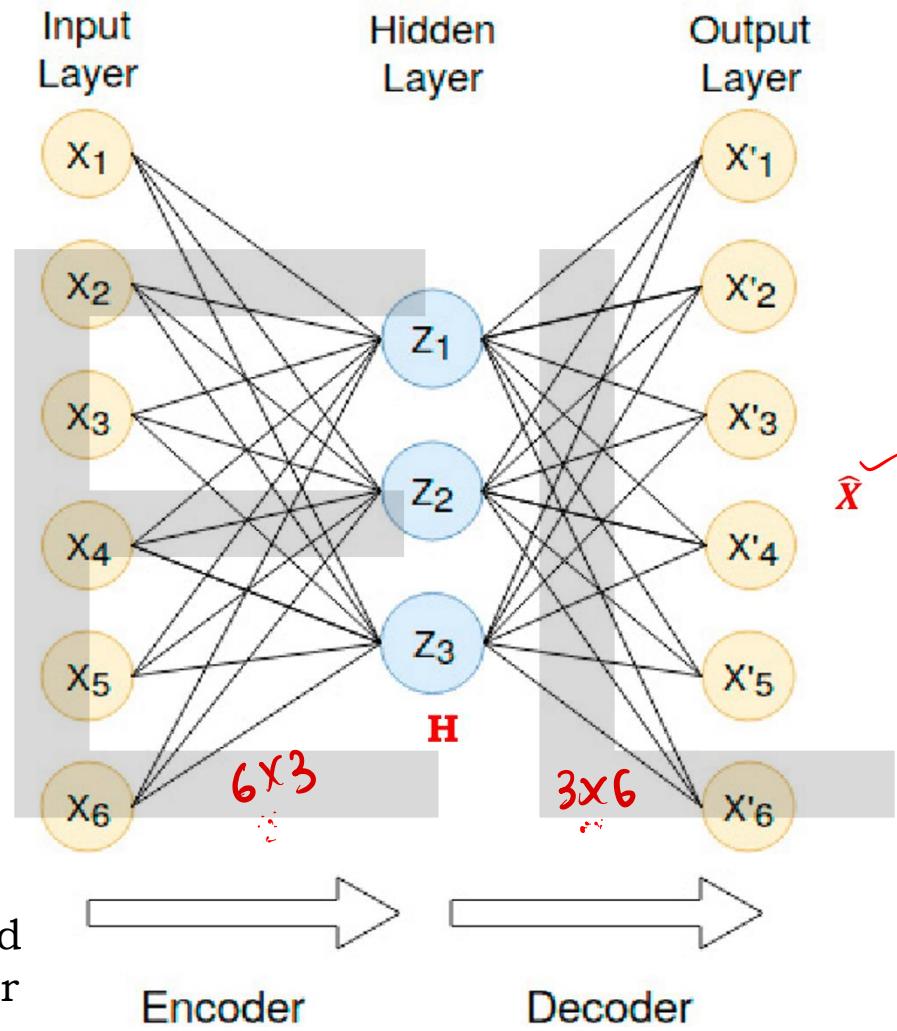
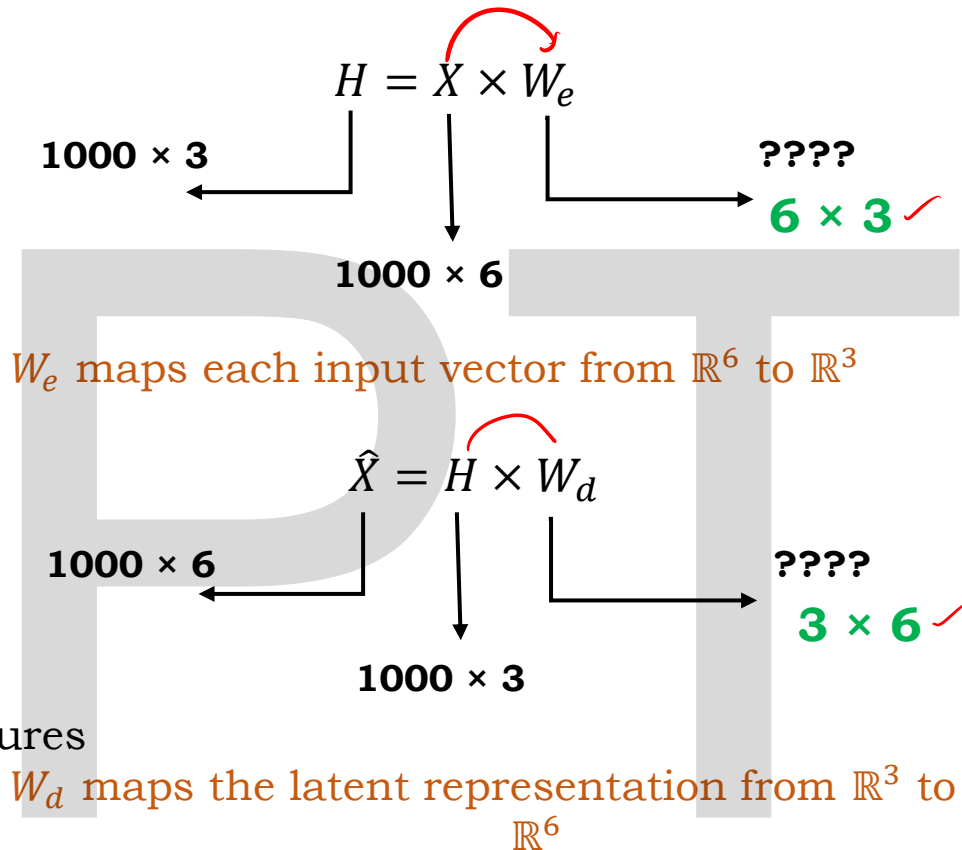
- ✓ Training objective: Reconstruction Loss Function (MSE, Binary Cross-Entropy)

NPTEL

Linear Autoencoder: Encoder and Decoder process

$H = g(X \times W_e)$
Where, $g(\cdot)$ is a activation function
H is the encoded data
X is the input
 W_e is Encoder Matrix

$\hat{X} = f(H \times W_d)$
Where,
H is the encoded data
 $\hat{X}$ is the reconstructed features
 W_d is Decoder Matrix
 $f(\cdot)$ is output activation



To achieve the encoding and decoding process, both W_e and W_d are learned during training, which increases the number of parameters and training time. This is called the **Generic Setting**.

$W_e = 6 \times 3$
 $W_d = 3 \times 6$

The decoder weight matrix has the reverse shape of the encoder matrix. Values can be anything.

then: $W_e: 6 \times 3$
 $W_e^T: 3 \times 6$
so: $W_d = W_e^T: 3 \times 6$

Only W_e to learn and the other is the Transpose of W_e . This is a commonly used and computationally efficient approach. **This is tied-weights assumption ($W_d = W_e^T$)**

Choice of Encoder and Decoder Activation Functions

Case 1: Binary Inputs

Suppose all input features are binary:

$$x_{ij} \in \{0,1\}$$

That means each element of the input vector is either 0 or 1.

The decoder output should also lie between 0 and 1. Therefore, the decoder activation is commonly chosen as **sigmoid**:

$$\hat{X} = f(H \times W_d)$$

where f is the **sigmoid activation function**.

$$f(a) = \frac{1}{1 + e^{-a}}$$

The encoder may also use a nonlinear activation such as **sigmoid or ReLU**:

$$H = g(X \times W_e)$$

Case 2: Real-Valued Inputs

Suppose the input features are real-valued:

$$x_{ij} \in \mathbb{R}$$

That means the input values can take continuous real values.

The decoder should reconstruct real-valued outputs. Therefore, the decoder activation is commonly chosen as **linear**:

$$\hat{X} = f(H \times W_d)$$

where f is the **linear activation function**.

$$f(a) = a$$

The encoder activation g can be chosen based on the model, commonly **ReLU, sigmoid, or tanh**:

$$H = g(X \times W_e)$$

Reconstruction Loss in Autoencoders: Mean Squared Error

Here, the inputs are real-valued. The main objective is to reconstruct the original input x_i , producing the reconstructed output $\hat{x}_i$.

Ideally, $\hat{x}_i = x_i$, **In practice**, $\hat{x}_i \approx x_i$ where x_i is the original data

The basic loss is to **minimize** the difference between original input X and the reconstructed $\hat{X}$

Then total loss:

$$\mathcal{L} = \frac{1}{m} \sum_{i=1}^m l_i$$

$$l_i = \|x_i - \hat{x}_i\|^2 \quad \text{Per-sample loss}$$

The total loss is the average reconstruction loss over all m samples.

$$f(W) = \min_W \|X - \hat{X}\|^2 \quad \text{Get } W \text{ which minimizes } \|X - \hat{X}\|^2 \quad X \in \mathbb{R}^{m \times n} \text{ be the input data matrix}$$

$$f(W) = \min_W \|X - HW_d\|^2 \quad \hat{X} = HW_d$$

$$W_e = W$$

$$W_d = W^T,$$

where W is the learnable quantity

$$f(W) = \min_W \|X - HW^T\|^2 \quad W_d = W^T$$

$$\mathbf{f(W)} = \min_W \|X - XWW^T\|^2 \quad \text{Reconstructed error}$$

$$f(W) = \min_W \|X - XW_e W^T\|^2 \quad H = XW_e$$

$$\mathbf{f(W)} = \min_W \|X - XWW^T\|^2 + \underbrace{\lambda \|W\|^2}_{\text{Regularization term}}$$

$$f(W) = \min_W \|X - XWW^T\|^2 \quad W_e = W$$

The value in the W matrix can be small or large. But always ensure W takes a small value. Therefore, **Regularization term**.

Once the training process is completed, we obtain the optimized weight matrix W^* , which minimizes the reconstruction loss.

Using the optimized weights, at encoder

$$XW^* = H^*$$

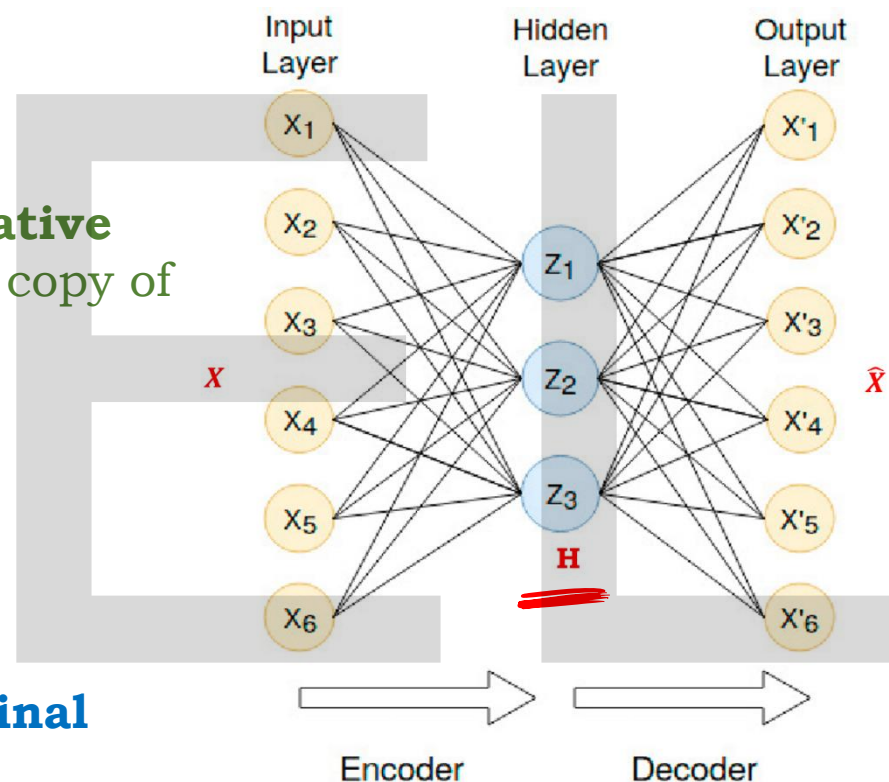
Thus, the latent representation H^* captures the **most rich informative structure of the data required for reconstruction**, not an exact copy of the input.

The reconstructed input is then obtained as:

$$H^* \cdot W^{*T} = \hat{X}$$

$$\|X - \hat{X}\|^2$$

the reconstructed output $\hat{X}$ is **a close approximation of the original input X** .



$$\underline{H^*} = X W^* \quad \hat{X} = H^* \cdot W^{*T}$$

Reconstruction Loss in Autoencoders: BCE

Here the inputs are binary data. The main idea is to reconstruct $\hat{x}_i$ at the end.

In the per-sample error, we are considering one input sample x_i .

Suppose one sample has d features:

$$x_i = [x_{i1}, x_{i2}, x_{i3}, \dots, x_{id}]$$

Here:

- i represents the sample number
- j represents the feature number
- d represents the total number of features in one sample

So,

$$l_i = - \sum_{j=1}^d [x_{ij} \log(\hat{x}_{ij}) + (1 - x_{ij}) \log(1 - \hat{x}_{ij})]$$

$$x_i = [1, 0, 1, 1, 0]$$

$$\hat{x}_i = [0.92, 0.08, 0.81, 0.76, 0.12]$$

Total loss

$$\mathcal{L}_{BCE} = \frac{1}{n} \sum_{i=1}^n l_i$$

Therefore,

$$\mathcal{L}_{BCE} = - \frac{1}{n} \sum_{i=1}^n \sum_{j=1}^d [x_{ij} \log(\hat{x}_{ij}) + (1 - x_{ij}) \log(1 - \hat{x}_{ij})]$$

For binary data:

$$x_{ij} \in \{0, 1\}$$

and the decoder output should be:

$$0 \leq \hat{x}_{ij} \leq 1$$

So we use sigmoid at the decoder output:

$$\hat{X} = \sigma(HW_d)$$

$\hat{x}_{ij}$ is a probability value between 0 and 1

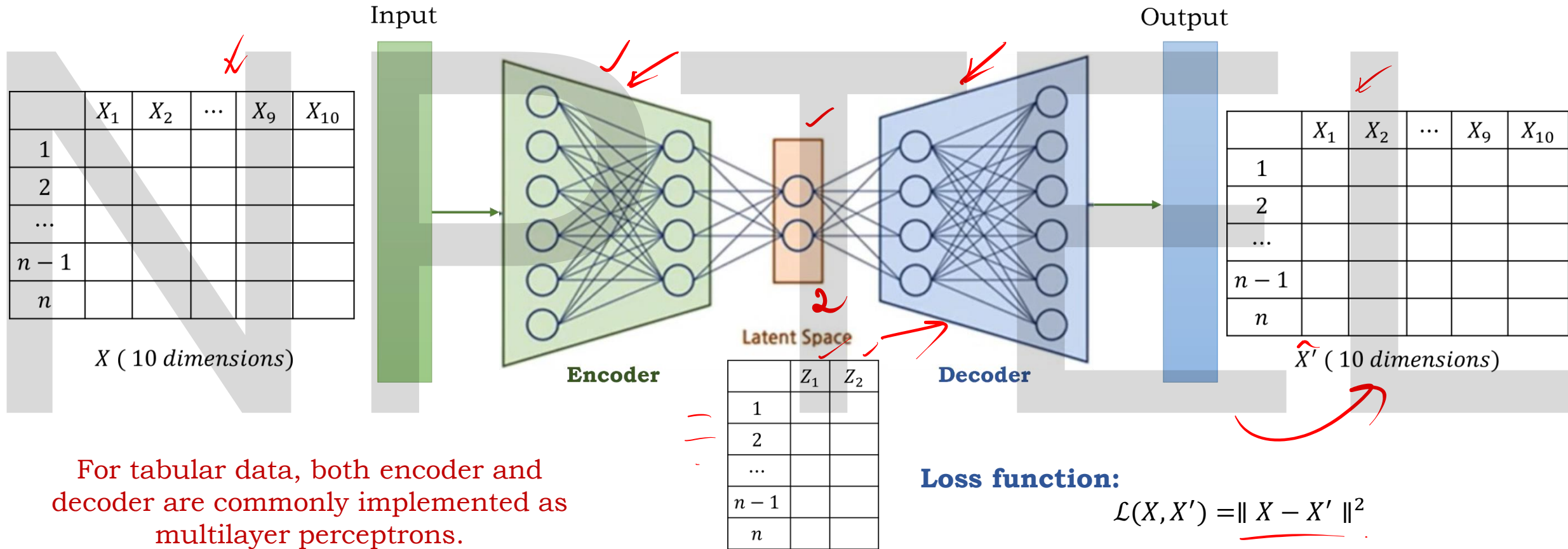
For any value strictly between 0 and 1, the logarithm is negative, and at 1 the log is 0.

So the inner expression gives a **negative value**. To convert it into a positive loss, we put a minus sign

$$f(W) = \min_W [\mathcal{L}_{BCE}(X, \hat{X})] + \underbrace{\lambda \|W\|^2}_{\text{Regularization term}}$$

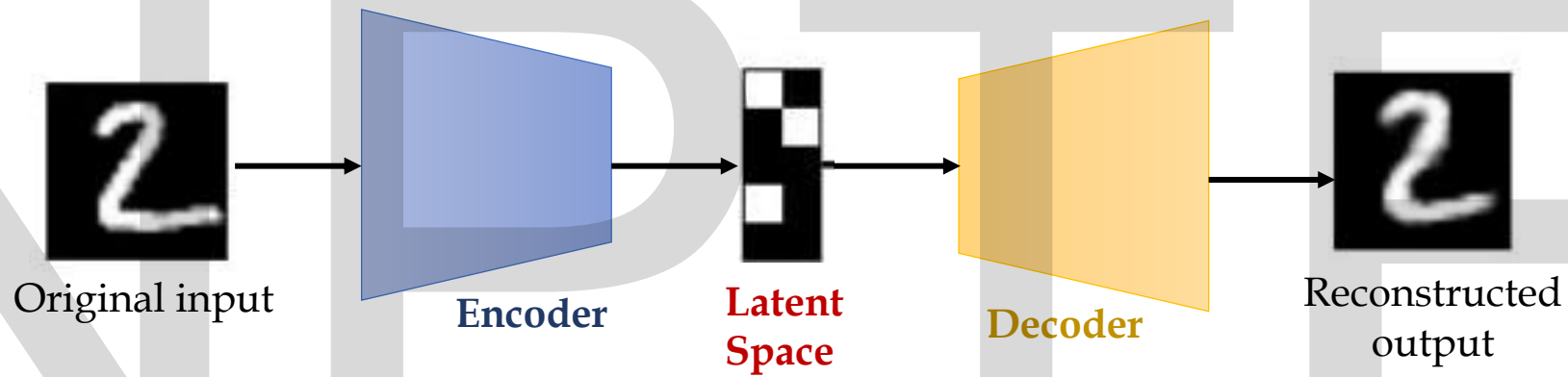
Regularization term

Autoencoder: Complete Learning Flow



$X \rightarrow \text{Encoder} \rightarrow Z \rightarrow \text{Decoder} \rightarrow X'$

The autoencoder is trained by minimizing the difference between the original input X and the reconstructed output X' .



For images, the encoder uses convolutional layers to extract spatial features, and the decoder uses upsampling/transposed convolution layers to reconstruct the image.

Next Session

Types of Autoencoders: A Practical Approach

- CNN Autoencoder
- Shallow Autoencoder
- Deep Autoencoder

NIPTEEL

Thank you